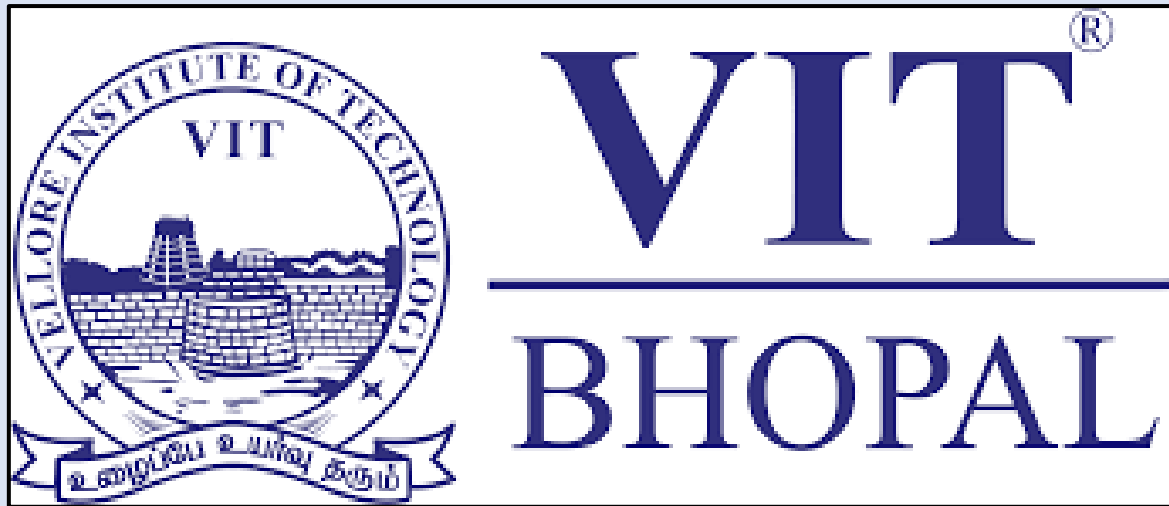




Campus

Management System

Course Name: Programming In Java

Course Code: CSE2006

Name: Saksham Gupta

Register No. : 25BAI10598

Branch: B. Tech C.S.E. (AI & ML)

Table Of Contents

Sr. No.	Topic
1.	Introduction
2.	Problem Statement
3.	Functional Requirements
4.	Non-functional Requirements
5.	System Architecture
6.	Design Diagrams
7.	Design Decisions & Rationale
8.	Implementation Details
9.	Screenshots / Results
10.	Testing Approach
11.	Challenges Faced
12.	Learnings & Key Takeaways
13.	Future Enhancements
14.	References

Introduction

The Campus Management System is a Java-based Student Academic and Support Management System designed to simplify the management of important student information in an educational institution. The system provides a simple and organized way to manage student details, attendance, marks, and student complaints.

In many institutions, maintaining student records manually can be time-consuming and may lead to errors or difficulty in accessing information. The Campus Management System addresses this problem by providing a centralized application where student data can be added, viewed, searched, updated, and stored digitally.

The project is developed using Core Java and demonstrates important programming concepts such as Object-Oriented Programming (OOP), classes and objects, encapsulation, ArrayList, file handling, validation, and exception handling. The system is divided into functional modules such as Student Management, Attendance Management, Marks Management, and Complaint Management.

The system also provides a student report that combines academic and attendance information in one place. Data is stored using Java file handling, allowing information to remain available even after the application is closed.

Overall, the Campus Management System provides a simple, user-friendly, and maintainable solution for basic student academic and support management while demonstrating the practical application of Java programming concepts.

Problem Statement

Managing student information manually in educational institutions can be time-consuming and difficult to maintain. Student details, attendance records, marks, and complaints are often handled separately, which can make it difficult to access and update information efficiently. Manual record keeping can also result in data entry errors, loss of information, and difficulty in tracking student academic performance and complaints.

To address these problems, there is a need for a simple and organized system that can store and manage student-related information digitally. The Campus Management System is developed to provide a centralized solution for managing student details, attendance, marks, and complaints.

The system allows users to add, view, search, update, and delete student information, record attendance, manage marks, submit and track complaints, and generate a basic student report. It also uses Java file handling to store data so that information can be accessed even after the application is closed.

The main purpose of this project is to reduce manual effort, improve data organization, and provide an easy-to-use system for basic student academic and support management.

Functional **Requirements**

The Campus Management System shall provide the following functional requirements:

1. Student Management

- The system shall allow the user to add new student details.
- The system shall store student ID, name, department, and email.
- The system shall display all registered students.
- The system shall allow the user to search for a student using the Student ID.
- The system shall allow the user to delete student records.
- The system shall prevent duplicate Student IDs.

2. Attendance Management

- The system shall allow the user to enter and update student attendance.
- The system shall record total classes and attended classes.

- The system shall calculate the attendance percentage automatically.
- The system shall display the attendance status.
- The system shall identify students with attendance below 75%.

3. Marks Management

- The system shall allow the user to enter and update marks for three subjects.
- The system shall validate that marks are between 0 and 100.
- The system shall calculate the student's average marks automatically.
- The system shall assign a grade based on the average marks.
- The system shall display the marks and calculated grade.

4. Complaint Management

- The system shall allow students to submit complaints.
- The system shall assign a unique Complaint ID automatically.

- The system shall store the student ID, complaint category, and description.
- The system shall display all submitted complaints.
- The system shall allow users to view complaints submitted by a particular student.
- The system shall allow the complaint status to be updated as Pending, In Progress, or Resolved.

5. Student Report

- The system shall generate a basic student report.
- The report shall display student details, attendance information, and marks.
- The system shall calculate and display academic performance information automatically.

6. Data Storage

- The system shall save student and complaint data to files.
- The system shall load previously saved data when the application starts.
- The system shall preserve data after the application is closed.

Non- Functional **Requirements**

The Campus Management System shall satisfy the following non-functional requirements:

1. Usability

- The system should have a simple and user-friendly menu-driven interface.
- Users should be able to perform operations easily without extensive technical knowledge.
- Clear messages should be displayed for successful operations and errors.

2. Performance

- The system should respond quickly to user inputs.
- Student records should be searched and displayed efficiently.
- The system should handle a reasonable number of student records without significant delay.

3. Reliability

- The system should store data correctly and consistently.

- Saved student and complaint information should remain available after the application is closed.
- The system should handle invalid inputs without crashing.

4. Error Handling

- The system should validate user inputs before processing them.
- Invalid marks, attendance values, and menu choices should be rejected.
- Appropriate error messages should be displayed to the user.

5. Maintainability

- The system should be divided into separate classes based on their responsibilities.
- The code should be simple, organized, and easy to understand.
- Comments and meaningful method names should be used where required.

6. Data Security

- Student and complaint data should be stored locally rather than being displayed unnecessarily.

- The system should prevent invalid or incomplete data from being stored.
- Data files should be accessed only by the application.

7. Scalability

- The system should allow additional features to be added in the future.
- New modules such as timetable management, fee management, or course management can be integrated without completely redesigning the system.

8. Resource Efficiency

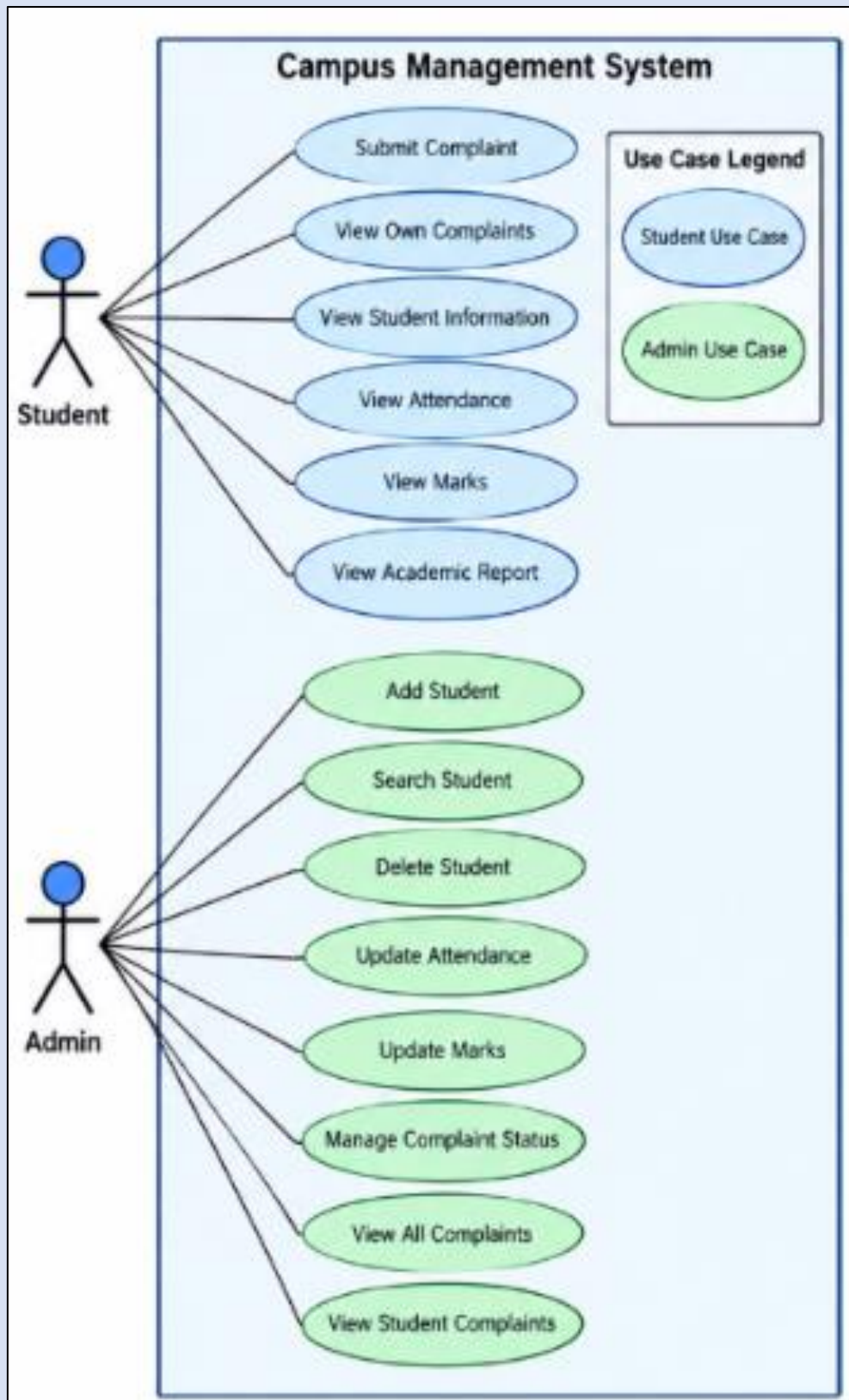
- The system should use memory and processing resources efficiently.
- The application should not require heavy external software or database services.
- Data should be stored using Java file handling to keep the system lightweight.

System Architecture

- **Presentation Layer:** Main.java provides the menu-driven interface and accepts user input.
- **Student Manager and Complaint Manager Layer:** Student, attendance, marks, and complaint classes handle the main operations and calculations.
- **Data Layer:** FileManager.java saves and loads data using Java file handling.

The system follows a simple modular structure, making it easier to understand and maintain.

Design Diagrams



Design Decisions and Rationale

- **Core Java:** Chosen because it is simple and sufficient for implementing the required features.
- **OOP:** Separate classes are used for students, attendance, marks, and complaints for better organization.
- **ArrayList:** Used to store multiple student and complaint records dynamically.
- **File Handling:** Used for simple and persistent data storage without requiring a database.
- **Menu-Driven Interface:** Provides a simple and easy-to-use console interface.
- **Input Validation:** Prevents invalid data such as marks above 100 or incorrect attendance values.
- **Modular Design:** Different classes handle different responsibilities, making the system easier to maintain and extend.

Implementation

Details

- Main.java: Provides the menu-driven interface and handles user input and program flow.
- Student.java: Stores student details such as ID, name, department, email, attendance, and marks.
- Attendance.java: Stores attendance details and calculates attendance percentage and status.
- Marks.java: Stores marks for three subjects and calculates average and grade.
- Complaint.java: Stores complaint details and maintains complaint status.
- StudentManager.java: Handles adding, searching, viewing, deleting students, and updating attendance and marks.
- ComplaintManager.java: Handles submitting, viewing, and updating complaints.
- FileManager.java: Provides file-based data storage by saving and loading student and complaint records.

Screenshots / Results

Student Management

```
===== MAIN MENU =====
1. Student Management
2. Attendance Management
3. Marks Management
4. Complaint Management
5. Student Report
6. Exit
=====
Enter your choice: 1

===== STUDENT MANAGEMENT =====
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Back
Enter your choice: 1

----- ADD STUDENT -----
Enter Student ID: 101
Enter Name: Saksham Gupta
Enter Department: C.S.E. (AI & ML)
Enter Email: sakshamcampus@gmail.com

Student added successfully.
```

Attendance Management

```
===== ATTENDANCE MANAGEMENT =====
1. Add / Update Attendance
2. View Attendance
3. Back
Enter your choice: 1
Enter Student ID: 101
Enter total classes: 36
Enter attended classes: 27

Attendance updated successfully.

===== ATTENDANCE MANAGEMENT =====
1. Add / Update Attendance
2. View Attendance
3. Back
Enter your choice: 2
Enter Student ID: 101

Student: Saksham Gupta
Total Classes : 36
Attended Classes: 27
Attendance : 75.00%
Status : Good
```

Marks Management

```
===== MARKS MANAGEMENT =====
1. Add / Update Marks
2. View Marks
3. Back
Enter your choice: 1
Enter Student ID: 101
Enter Subject 1 marks: 99
Enter Subject 2 marks: 98
Enter Subject 3 marks: 97

Marks updated successfully.

===== MARKS MANAGEMENT =====
1. Add / Update Marks
2. View Marks
3. Back
Enter your choice: 2
Enter Student ID: 101

Student: Saksham Gupta
Subject 1 : 99.0
Subject 2 : 98.0
Subject 3 : 97.0
Average : 98.00
Grade : A+
```

Complaint Management

```
===== COMPLAINT MANAGEMENT =====
1. Submit Complaint
2. View All Complaints
3. View Student Complaints
4. Update Complaint Status
5. Back
Enter your choice: 2

===== ALL COMPLAINTS =====

-----
Complaint ID : 1
Student ID : 123
Category : Civil
Description : Cracks in wall of the hostel room
Status : Resolved
-----

-----
Complaint ID : 2
Student ID : 101
Category : Civil
Description : AC not working
Status : Pending
-----
```

Student Report

===== STUDENT REPORT =====

Enter Student ID: 101

=====

STUDENT REPORT

=====

Student ID : 101
Name : Saksham Gupta
Department : C.S.E. (AI & ML)
Email : sakshamcampus@gmail.com

--- Attendance ---

Total Classes : 36
Attended Classes: 27
Attendance : 75.00%
Status : Good

--- Marks ---

Subject 1 : 99.0
Subject 2 : 98.0
Subject 3 : 97.0
Average : 98.00
Grade : A+

=====

Testing Approach

The project was tested manually by checking each module with valid and invalid inputs.

- Student details were tested for adding, searching, and deleting.
- Attendance and marks were tested with different values.
- Invalid inputs were checked to ensure proper error messages.
- Complaint submission and status updates were tested.
- File storage was tested by restarting the program and checking saved data.
- The complete system was tested through the main menu to verify the overall workflow.

Challenges Faced

- **Data Management:** Managing student and complaint records using ArrayList required proper handling of adding, searching, and deleting data.
- **File Handling:** Implementing file storage and loading using Java serialization required careful exception handling.
- **Input Validation:** Handling invalid user inputs, marks, attendance values, and menu choices was challenging.
- **Class Integration:** Connecting the different classes and ensuring they worked together correctly required careful design.
- **Calculations:** Implementing correct attendance percentages, marks averages, and grade calculations required proper validation and logic.

Learning & Key Takeaways

- Learned how to apply **Object-Oriented Programming** concepts in a practical Java project.
- Gained experience in designing and connecting multiple Java classes.
- Learned to use **ArrayList** for managing student and complaint records.
- Improved understanding of **file handling and data persistence** using Java serialization.
- Learned how to implement **input validation and exception handling**.
- Gained experience in implementing calculations such as attendance percentage, marks average, and grades.
- Improved problem-solving, debugging, and software development skills.

Future

Enhancements

- **Database Integration:** Replace file storage with MySQL or another database for better data management.
- **Graphical User Interface:** Develop a GUI or web interface for easier interaction.
- **Login System:** Add secure login and role-based access for students and administrators.
- **Timetable Management:** Add features for managing class schedules and timetables.
- **Fee Management:** Include student fee records and payment tracking.
- **Notifications:** Add notifications for low attendance, marks, and complaint updates.
- **Advanced Reports:** Generate detailed academic and attendance reports.

References

Oracle Java Documentation – Used for understanding Java classes, objects, methods, and basic programming concepts.

Herbert Schildt, *Java: The Complete Reference*, McGraw Hill.

E. Balagurusamy, *Programming with Java: A Primer*, McGraw Hill